Практична робота №6

Тема: Навігація та пошук елементів у DOM-дереві.

Мета: Навчитися здійснювати пошук елементів та зміна вмісту та властивостей.

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

DOM (Document Object Model) представляє HTML-документ як дерево тегів, де кожен вузол − це об'єкт. Для навігації між ними існують спеціальні властивості.

Зв'язки між вузлами.

Всі вузли мають батьківські, дочірні та сусідні елементи:

- *Батько*: `parentNode` (будь-який вузол) або `parentElement` (тільки якщо батько − тег).
- *Діти:* `childNodes` (всі вузли, включно з текстом), `children` (тільки теги).
- *Перший/Останній*: `firstChild/lastChild` або `firstElementChild/lastElementChild`.
- *Сусіди:* `nextElementSibling` та `previousElementSibling`.

Для того щоб чітко зрозуміли різницю між вузлами (`nodes`) та елементами (`elements`), найкраще продемонструвати це на прикладі структури, де є переноси рядків та пробіли, адже саме вони стають «невидимими» текстовими вузлами.

HTML-структура для прикладу

```
<div id="container">
    <p id="first-p">Перший абзац</p>
    <p id="second-p">Другий абзац</p>
</div>
```

а. Батьківські елементи

Різниця між `parentNode` та `parentElement` помітна лише на «верхівці» документа.

JavaScript

```
const p = document.querySelector('#first-p');

console.log(p.parentNode);    // <div id="container">
console.log(p.parentElement); // <div id="container"> (те саме)

// Різниця:
console.log(document.documentElement.parentNode);    // document
(вузол документа)
console.log(document.documentElement.parentElement);  //  null  (бо
document — це не тег/елемент)
```

b. *Дочірні елементи (Діти)*

Це найважливіший момент: `childNodes` бачить пробіли та переноси рядків як текстові вузли.

JavaScript

```
const container = document.querySelector('#container');

console.log(container.childNodes);
// Поверне: [text, comment, text, p#first-p, text, p#second-p,
text]
// (text — це переноси рядків між тегами)

console.log(container.children);
// Поверне: [p#first-p, p#second-p] (тільки теги)
```

c. *Перший та Останній вузли*

Тут найчастіше припускаються помилок, намагаючись отримати перший тег через `firstChild`.

JavaScript

```
const container = document.querySelector('#container');

console.log(container.firstChild);        // #text (перенос рядка
після <div>)
console.log(container.firstElementChild); // <p id="first-p">
(перший тег)

console.log(container.lastChild);         // #text (перенос рядка
перед </div>)
console.log(container.lastElementChild);  // <p id="second-p">
```

d. *Сусіди (Сиблінги)*

Навігація між елементами на одному рівні.

JavaScript

```
const secondP = document.querySelector('#second-p');

// Навігація вгору/вниз по сусідах
console.log(secondP.previousElementSibling); // <p id="first-p">
console.log(secondP.nextElementSibling);     // null (бо далі
немає тегів)

// Вузли-сусіди (враховуючи текст)
console.log(secondP.previousSibling); // #text (пробіл між
абзацами)
```

Підсумовуюча таблиця

Тип навігації	**Враховує все (вузли)**	**Тільки теги (елементи)**
Батько	`parentNode`	`parentElement`
Діти	`childNodes`	`children`

Перший	`firstChild`	`firstElementChild`
Останній	`lastChild`	`lastElementChild`
Попередній	`previousSibling`	`previousElementSibling`
Наступний	`nextSibling`	`nextElementSibling`

Примітка: У 90% випадків у веб-розробці ми використовуємо властивості зі словом `Element`, щоб випадково не змінити стиль текстового вузла або коментаря

Методи пошуку елементів

Сучасний стандарт JavaScript виділяє два основні методи:

- `querySelector(selector)` – повертає **перший** елемент, що відповідає CSS-селектору.
- `querySelectorAll(selector)` – повертає статичну **колекцію** всіх знайдених елементів.

Для демонстрації різниці між цими методами використаємо приклад зі списком, де декілька елементів мають однаковий клас

HTML-структура для прикладу

```
<ul class="nav">
    <li class="nav-item">Головна</li>
    <li class="nav-item active">Новини</li>
    <li class="nav-item">Контакти</li>
</ul>
```

`querySelector(selector)`

Цей метод працює за принципом **«знайти перший і зупинитися»**. Навіть якщо на сторінці 100 елементів із таким класом, ви отримаєте лише один об'єкт.

JavaScript

```
// Шукаємо перший елемент із класом .nav-item
const firstItem = document.querySelector('.nav-item');

console.log(firstItem.textContent);
// Виведе: "Головна" (тільки перший знайдений)

// Можна використовувати будь-які CSS-селектори
const activeItem = document.querySelector('.nav-
item.active');
activeItem.style.color = 'red'; // Змінить колір тільки
"Новини"
```

`querySelectorAll(selector)`

Цей метод повертає **NodeList** (статичну колекцію) усіх елементів. Щоб змінити ці елементи, їх потрібно перебрати за допомогою циклу (наприклад, `forEach`).

JavaScript

```
// Шукаємо ВСІ елементи з класом .nav-item
const allItems = document.querySelectorAll('.nav-item');

console.log(allItems.length);
// Виведе: 3

// Спроба змінити колір всім одразу через allItems.style
не спрацює!
// Потрібно використовувати цикл:
allItems.forEach((item, index) => {
    item.textContent = `${index + 1}.
${item.textContent}`;
});
/* Результат у списку:
1. Головна
2. Новини
3. Контакти
*/
```

Порівняльна таблиця

Характеристика	querySelector	querySelectorAll
Що повертає	Один елемент (об'єкт) або `null`	Колекцію `NodeList` (масивоподібний об'єкт)
Поведінка	Зупиняється після першої знахідки	Проходить крізь весь документ
Доступ до стилів	Прямий: `el.style.color = 'red'`	Тільки через цикл або індекс: `els[0].style...`
Коли обирати	Коли елемент унікальний або потрібен лише перший	Коли треба змінити групу однотипних елементів

Важливо: Зверніть увагу, що `querySelectorAll` повертає **статичну** колекцію. Це означає, що якщо ви видалите елемент зі сторінки після виконання пошуку, він все одно залишиться в цій змінній JavaScript до кінця роботи скрипта.

Залишаються актуальними:

- `getElementById('id')` – найшвидший пошук за ідентифікатором.
- `closest(selector)` – шукає найближчого предка (або самого себе), що відповідає селектору.

`document.getElementById()`

Якщо у елемента є атрибут id, то ми можемо отримати його викликом `document.getElementById(id)`, де б він не знаходився.

Наприклад:

```
<div id="elem">
  <div id="elem-content">Element</div>
</div>
<script>
  // отримати елемент
  let elem = document.getElementById('elem');
  // зробити фон червоним
  elem.style.background = 'red';
</script>
```

Метод `closest()`

Метод **`element.closest(selector)`** шукає найближчого предка (або самого себе), який відповідає вказаному CSS-селектору.

Принцип роботи:

1. Перевіряється сам елемент, на якому викликано метод.
2. Якщо він не відповідає селектору, пошук переходить до батьківського елемента.
3. Пошук йде вгору по дереву DOM доти, доки не знайде збіг або не досягне `null` (верхівки документа).

Приклад застосування

Умова. Представимо складну структуру картки товару, де кнопка знаходиться глибоко всередині вкладених тегів:

HTML-структура картки

```
<div class="product-card" data-id="105">
    <div class="card-body">
        <h3 class="title">Смартфон</h3>
```

```
        <div class="actions">
            <button class="buy-btn">
                <span class="icon">🛒</span>
                <span class="text">Купити</span>
            </button>
        </div>
    </div>
</div>
```

Завдання. Припустимо, нам потрібно отримати ID картки (`data-id`) при кліку на кнопку або навіть на іконку всередині кнопки.

JavaScript

```
 const btn = document.querySelector('.buy-btn');

 btn.addEventListener('click', function(event) {
     // Навіть якщо клікнули на <span> всередині кнопки,
     // closest('.product-card') знайде головний
контейнер
     const card = event.target.closest('.product-card');

     if (card) {
         const productId = card.dataset.id;
         console.log(`Ви додали у кошик товар з ID:
${productId}`);

         // Можна також підсвітити всю картку
         card.style.border = "2px solid green";
     }
 });
```

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

Завдання 1. Модифікація унікальних елементів

***Постановка завдання*:** Ви отримали дані з сервера про користувача. Потрібно заповнити «картку профілю» на сторінці, використовуючи ідентифікатори.

HTML

```
<div id="user-profile">
    <h2 id="user-name">Гість</h2>
    <p id="user-status">Статус: невідомий</p>
    <div id="user-avatar" style="width: 100px; height:
100px; background: grey;"></div>
```

```
</div>
```

Завдання на JavaScript:

1. За допомогою `document.getElementById()` знайдіть заголовок `user-name` і змініть його текст на ваше прізвище та ім'я.
2. Знайдіть елемент `user-status` і змініть його текст на "Студент 2-го курсу". Також змініть його колір на зелений (властивість `style.color`).
3. Знайдіть блок `user-avatar` і змініть його фоновий колір (`style.backgroundColor`) на синій, а також додайте йому рамку.

Завдання 2. Навігація по структурі.

Створіть HTML-сторінку з меню:

HTML

```
<ul id="main-menu">
  <li>Головна</li>
  <li class="active">Послуги</li>
  <li>Контакти</li>
</ul>
```

Напишіть код, який:

1. Отримає елемент `li` з класом `active`.
2. Виведе в консоль текст його попереднього сусіда та наступного сусіда.
3. Змінить колір фону батьківського елемента (`ul`).

Завдання 3. Пошук та маніпуляція

Дано список товарів/

HTML

```
<div class="catalog">
  <div class="item">Товар 1 <button>Купити</button></div>
  <div class="item">Товар 2 <button>Купити</button></div>
  <div class="item">Товар 3 <button>Купити</button></div>
</div>
```

Завдання:

1. Виберіть усі кнопки «Купити» за допомогою `querySelectorAll`.
2. За допомогою циклу `forEach` змініть текст кожної другої кнопки на “В кошику”.
3. Додайте клас `highlight` до першого елемента списку.

Завдання 4. Динамічний пошук (Робота з closest)

Реалізуйте логіку для таблиці:

1. Створіть таблицю 3x3.
2. При кліку на будь-яку комірку (`td`), знайдіть її рядок-предок (`tr`) за допомогою методу `closest` і змініть йому колір фону.